

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Materia: Programación I

**Gestión de Datos de Países en
Python:
filtros, ordenamientos y
estadísticas**

Trabajo Práctico Integrador (TPI)

Integrantes:

Franco Elian Enrici, y Franco
Exequiel Herrera Schachinger.

Fecha de entrega:

17 de junio de 2025

Tabla de contenido

1	Introducción.....	4
1.1	Objetivo del Trabajo.....	4
2	Marco Teórico.....	4
2.1	Listas.....	4
2.2	Diccionarios.....	5
2.3	Funciones.....	5
2.4	Estructuras Condicionales.....	6
2.5	Estructuras Repetitivas.....	7
2.7	Ordenamiento.....	8
2.8	Estadísticas Básicas.....	9
3	Diseño del Sistema.....	9
3.1	Diagrama de Flujo.....	9
3.2	Estructura del Proyecto.....	10
4	Desarrollo de las Funcionalidades.....	11
4.1	Gestión de Datos.....	11
4.2	Búsquedas.....	12
4.3	Filtros.....	13
4.4	Ordenamientos.....	13
4.5	Estadísticas.....	14
5	Validaciones y Manejo de Errores.....	15
6	Evidencias de Funcionamiento.....	16
6.1	Agregar País.....	16
6.2	Actualizar País.....	16
6.3	Buscar País.....	17
6.3	Filtrar Países.....	17
6.4	Ordenar Países.....	18
6.5	Estadísticas.....	18
6.6	Guardar Datos.....	19
7	Dificultades Encontradas.....	19
8	Conclusión.....	20
9	Anexos.....	21

9.1	Enlace al repositorio GitHub.....	21
9.2	Enlace al video demostración.....	21
10.	Referencias.....	21

1 Introducción

El presente trabajo tiene como objetivo desarrollar una aplicación en lenguaje Python destinada a gestionar información detallada de países. Esta gestión incluye la manipulación de datos mediante estructuras como listas y diccionarios, así como la implementación de funciones que facilitan filtros, ordenamientos y cálculos estadísticos básicos. La aplicación permite almacenar y actualizar información para ofrecer herramientas útiles de búsqueda y análisis, promoviendo la modularización y reutilización eficiente del código fuente.

Este proyecto está enmarcado en la materia Programación I de la Tecnicatura Universitaria en Programación a Distancia y busca fortalecer competencias en el manejo de estructuras de datos y lógica de programación aplicada.

1.1 Objetivo del Trabajo

Desarrollar una aplicación en Python capaz de gestionar información de países mediante el uso de listas, diccionarios, funciones, filtros, ordenamientos y estadísticas, permitiendo la lectura y persistencia de datos a través de archivos CSV.

2 Marco Teórico

2.1 Listas

Las listas son estructuras de datos ordenadas y mutables que permiten almacenar múltiples elementos dentro de una misma variable (Python Software Foundation, 2025). En Python se representan mediante corchetes (`[]`) y son ampliamente utilizadas para gestionar colecciones de datos.

En el presente proyecto se utilizó una lista principal para almacenar la información de los países cargados desde el archivo CSV. Cada elemento de dicha lista corresponde a un diccionario que representa un país y contiene información como nombre, población, superficie y continente.

El uso de listas permitió recorrer fácilmente los registros mediante estructuras repetitivas, realizar búsquedas, aplicar filtros, efectuar ordenamientos y calcular estadísticas sobre el conjunto de datos almacenados. Gracias a esta estructura fue posible gestionar de manera eficiente todos los países dentro del sistema.

Ejemplo básico de una lista:

```
países = ["Argentina", "Brasil", "Chile"]
```

2.2 Diccionarios

Los diccionarios son estructuras de datos que permiten almacenar información mediante pares clave-valor, facilitando el acceso eficiente a los datos y su organización estructurada (Python Software Foundation, 2025). En Python se representan utilizando llaves (`{}`), donde cada clave identifica de manera única un dato asociado a un valor determinado.

En este proyecto se utilizaron diccionarios para representar cada país del sistema. Cada diccionario contiene las claves **nombre**, **población**, **superficie** y **continente**, permitiendo organizar la información de forma clara y estructurada.

El uso de diccionarios facilitó el acceso a los datos mediante sus claves. Por ejemplo, para consultar la población de un país es posible acceder directamente al valor asociado a la clave correspondiente, lo que mejora la legibilidad y comprensión del código. Además, esta estructura permite agregar nuevos atributos de manera sencilla, como capital, idioma o moneda, sin necesidad de modificar significativamente la lógica del programa.

Gracias a su flexibilidad y facilidad de acceso a los datos, los diccionarios resultaron fundamentales para la implementación de las búsquedas, filtros, ordenamientos y estadísticas desarrolladas en el sistema.

Ejemplo básico:

```
pais = {"nombre": "Argentina", "poblacion": 45000000,  
        "superficie": 2780000}
```

2.3 Funciones

Las funciones son bloques de código diseñados para realizar una tarea específica. Su principal objetivo es dividir un problema complejo en partes más pequeñas y fáciles de comprender, implementar y mantener.

En este proyecto se utilizó una estructura modular basada en funciones, donde cada

una posee una única responsabilidad. Por ejemplo, existen funciones destinadas a agregar países, realizar búsquedas, aplicar filtros, generar estadísticas y validar datos ingresados por el usuario.

La utilización de funciones permitió mejorar la organización del código, facilitar la detección y corrección de errores, y aumentar la legibilidad del programa. Además, la modularización permitió distribuir el desarrollo en distintos archivos, favoreciendo el trabajo colaborativo y permitiendo reutilizar funcionalidades mediante la importación de módulos en el archivo principal.

Gracias a este enfoque, el sistema resulta más escalable, mantenible y fácil de comprender, tanto para los desarrolladores como para futuros usuarios que deseen analizar o modificar el código.

2.4 Estructuras Condicionales

Las estructuras condicionales permiten que un programa tome decisiones según determinadas condiciones. Estas estructuras permiten controlar el flujo de ejecución de un programa mediante la evaluación de expresiones lógicas (Python Software Foundation, 2025). En Python se implementan principalmente mediante las sentencias `if`, `elif` y `else`, las cuales evalúan expresiones lógicas y ejecutan diferentes acciones dependiendo del resultado obtenido.

En el desarrollo de este proyecto, los condicionales fueron fundamentales para controlar el flujo de ejecución del sistema. Se utilizaron en la navegación de los menús, en la validación de opciones ingresadas por el usuario, en las búsquedas de países y en la generación de estadísticas.

Además, permitieron verificar situaciones específicas, como la existencia de un país dentro de la lista, la selección de criterios de ordenamiento o filtrado, y la detección de entradas inválidas. Gracias a estas estructuras fue posible brindar mensajes claros al usuario y garantizar un funcionamiento correcto del sistema ante distintos escenarios. Por su capacidad para controlar la lógica del programa y responder a diferentes situaciones, las estructuras condicionales constituyen uno de los elementos más importantes utilizados en el desarrollo de la aplicación.

2.5 Estructuras Repetitivas

Las estructuras repetitivas permiten ejecutar un conjunto de instrucciones múltiples veces sin necesidad de escribir el mismo código de forma repetida. Estos bucles permiten automatizar tareas repetitivas y procesar conjuntos de datos de manera eficiente dentro de una aplicación (Python Software Foundation, 2025). En Python, las más utilizadas son los bucles `for` y `while`.

En este proyecto, el bucle `for` fue fundamental para recorrer la lista de países y acceder a la información almacenada en cada registro. Gracias a esta estructura fue posible realizar búsquedas, aplicar filtros, calcular estadísticas y mostrar información al usuario de manera eficiente.

Por otra parte, el bucle `while` se utilizó principalmente para controlar el funcionamiento de los menús del sistema, permitiendo que el programa permanezca en ejecución hasta que el usuario decida finalizarlo. Asimismo, se empleó en procesos de validación para solicitar nuevamente una entrada cuando los datos ingresados no cumplían con las condiciones establecidas.

La utilización de estructuras repetitivas permitió automatizar tareas, reducir la cantidad de código y facilitar el procesamiento de grandes cantidades de información dentro del sistema.

2.6 Archivos CSV

Un archivo CSV (Comma-Separated Values o Valores Separados por Comas) es un formato de texto plano utilizado para almacenar datos tabulares de manera sencilla y estructurada. Cada línea representa un registro y los distintos campos se encuentran separados por comas, lo que facilita su lectura y procesamiento mediante programas informáticos.

Justamente, los archivos CSV constituyen uno de los formatos más utilizados para el intercambio y almacenamiento de datos tabulares debido a su simplicidad y compatibilidad con múltiples herramientas de análisis (McKinney, 2017).

En este proyecto se utilizaron archivos CSV para almacenar la información de los países de forma persistente. Esta decisión permitió mantener los datos separados del código fuente, facilitando su modificación, actualización y reutilización sin necesidad de realizar cambios en la programación de la aplicación.

La función de carga de datos fue responsable de leer el archivo CSV al iniciar el sistema y transformar cada registro en un diccionario dentro de una lista. Por otro lado, la función de guardado permitió almacenar en el archivo las modificaciones realizadas por el usuario, como la incorporación de nuevos países o la actualización de información existente.

El uso de archivos CSV proporcionó una solución simple y eficiente para gestionar los datos del proyecto, permitiendo conservar la información entre distintas ejecuciones del programa.

2.7 Ordenamiento

Los ordenamientos son técnicas utilizadas en programación para organizar un conjunto de datos siguiendo un criterio determinado. Su principal objetivo es facilitar la visualización, búsqueda y análisis de la información almacenada. Mediante estos procesos, los datos pueden presentarse de forma ascendente o descendente según una característica específica, permitiendo una interpretación más clara y eficiente de los registros.

En Python existen diversas formas de realizar ordenamientos. Una de las más utilizadas es la función integrada `sorted()`, que permite generar una nueva colección ordenada a partir de una secuencia de datos. Esta función admite la utilización del parámetro `key`, el cual especifica el criterio de comparación que será utilizado durante el proceso de ordenamiento. Asimismo, mediante el parámetro `reverse` es posible invertir el orden de los resultados para obtener una clasificación descendente.

En el presente proyecto se implementaron funcionalidades que permiten ordenar los países según diferentes criterios, tales como el nombre, la población y la superficie territorial. Estas operaciones facilitan la consulta de información y permiten identificar rápidamente países con determinadas características. Por ejemplo, el ordenamiento por población posibilita visualizar cuáles son los países más poblados, mientras que el ordenamiento por superficie permite comparar la extensión territorial de cada registro.

La aplicación de técnicas de ordenamiento contribuye significativamente a mejorar la organización de los datos y constituye una herramienta fundamental para el análisis y procesamiento de información dentro de los sistemas informáticos.

2.8 Estadísticas Básicas

Las estadísticas básicas permiten obtener información relevante a partir de un conjunto de datos, facilitando su análisis e interpretación. Los cálculos estadísticos básicos permiten resumir conjuntos de datos mediante indicadores representativos que facilitan su interpretación y análisis (McKinney, 2017). A través de diferentes cálculos es posible identificar tendencias, comparar registros y obtener indicadores representativos de la información almacenada.

En este proyecto se implementaron diversas estadísticas sobre los países registrados, incluyendo la identificación del país con mayor y menor población, el cálculo del promedio de población, el promedio de superficie y la cantidad de países pertenecientes a cada continente.

Estas funcionalidades permiten al usuario obtener una visión general de los datos disponibles y realizar comparaciones de manera rápida y sencilla. Por ejemplo, conocer el promedio de población ayuda a comprender las características generales del conjunto de países, mientras que identificar los valores máximos y mínimos facilita el análisis de casos particulares.

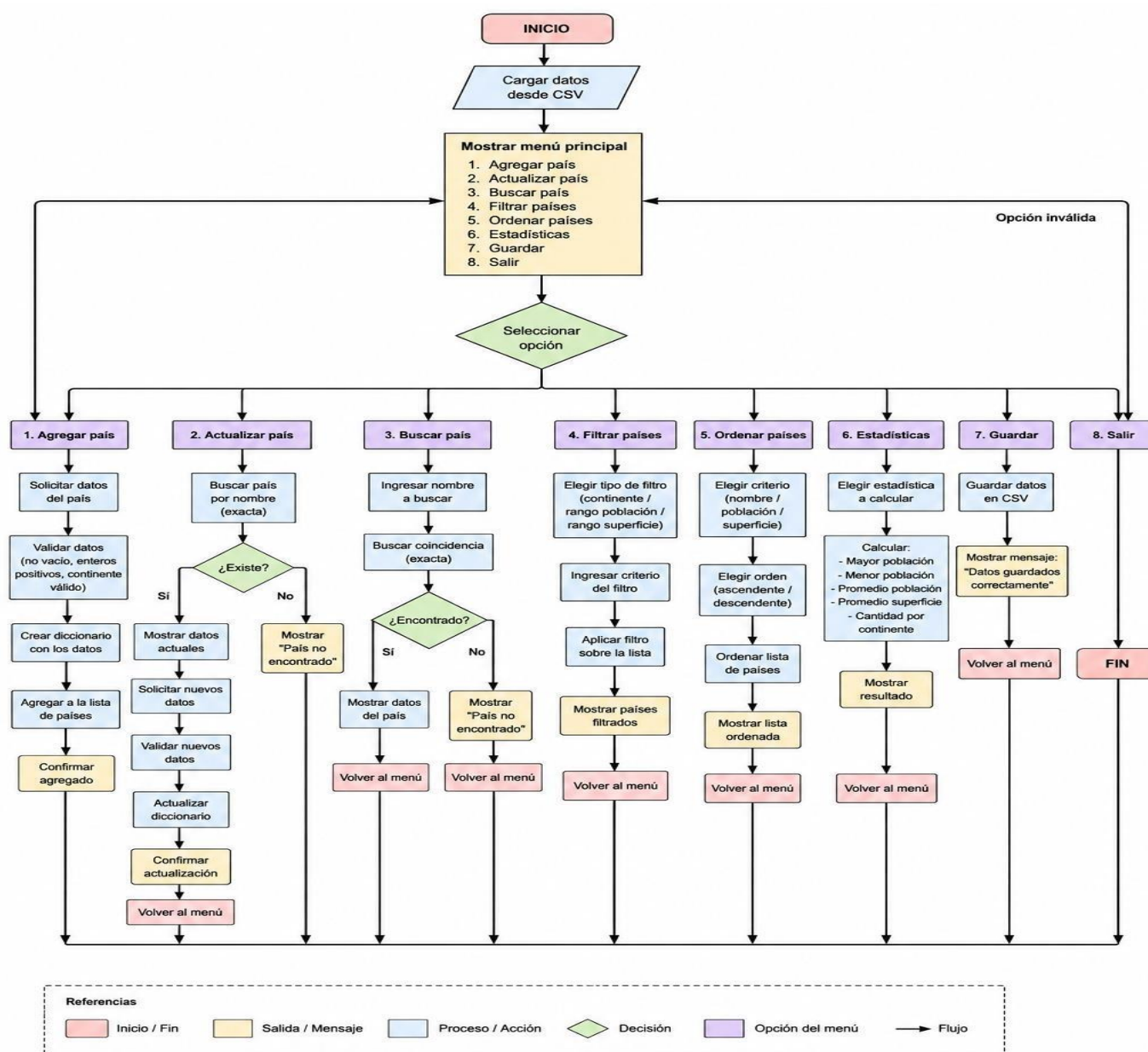
La incorporación de estadísticas básicas aportó valor al sistema, transformando los datos almacenados en información útil para la consulta y el análisis de países, facilitando su manipulación externa.

3 Diseño del Sistema

3.1 Diagrama de Flujo

El sistema fue desarrollado siguiendo una arquitectura modular, donde cada componente posee una responsabilidad específica dentro de la aplicación. Esta organización permitió dividir el problema en partes más pequeñas y manejables, favoreciendo la legibilidad del código, la detección de errores y el mantenimiento futuro del proyecto.

La interacción entre los módulos se realiza a través del archivo principal, encargado de coordinar las distintas funcionalidades del sistema. De esta manera, las operaciones de gestión de datos, búsquedas, filtros, ordenamientos, estadísticas y validaciones permanecen separadas, reduciendo el acoplamiento entre componentes y facilitando el trabajo colaborativo entre los integrantes del equipo.



3.2 Estructura del Proyecto

El proyecto fue desarrollado utilizando una arquitectura modular, separando las distintas funcionalidades en archivos independientes según su responsabilidad. Esta organización permitió mantener el código ordenado, mejorar su legibilidad y facilitar tanto el mantenimiento como la detección de errores.

La división en módulos también favoreció el trabajo colaborativo, permitiendo que los integrantes desarrollaran diferentes partes del sistema de forma independiente y posteriormente integraran los cambios mediante Git y GitHub.

Archivo	Responsabilidad
main.py	Controla el flujo principal del programa, gestiona el menú y la interacción con el usuario.
datos.py	Maneja la carga y almacenamiento de datos en memoria, así como la lectura y escritura del archivo CSV.
busquedas.py	Implementa funciones para búsquedas exactas y parciales en los datos de países.
filtros.py	Contiene las funciones que posibilitan filtrar la información por continente, población y superficie.
ordenamientos.py	Define los algoritmos para ordenar listas de países según diferentes criterios y órdenes.
estadisticas.py	Proporciona cálculos estadísticos básicos como promedios, máximos, mínimos y conteos por continente.
validaciones.py	Incluye funciones para validar entradas de usuario, asegurar tipos y rangos correctos, y manejo de errores.
paises.csv	Archivo de datos que almacena la información persistente de los países en formato CSV.

4 Desarrollo de las Funcionalidades

4.1 Gestión de Datos

La gestión de datos constituye la base del sistema, ya que permite almacenar, modificar y persistir la información de los países registrados. Al iniciar la aplicación, los datos son cargados desde un archivo CSV que contiene un conjunto inicial de países. Cada registro leído se transforma en un diccionario con los atributos nombre, población, superficie y continente.

Todos los diccionarios son almacenados dentro de una lista, que actúa como estructura principal de trabajo durante la ejecución del programa. Esta organización facilita el acceso, recorrido y manipulación de la información mediante las distintas funcionalidades implementadas.

La función de agregado permite incorporar nuevos países al sistema, solicitando al usuario los datos necesarios y creando un nuevo diccionario que posteriormente se agrega a la lista. Por otro lado, la función de actualización posibilita modificar la información de un país existente, manteniendo los datos actualizados durante la ejecución del programa.

Los cambios realizados no se almacenan de manera inmediata en el archivo CSV, sino que permanecen en memoria hasta que el usuario selecciona la opción de guardado. En ese momento, toda la información actualizada se escribe nuevamente en el archivo, garantizando la persistencia de los datos entre distintas ejecuciones.

Asimismo, se incorporó manejo básico de errores para la carga del archivo CSV, informando al usuario cuando el archivo no puede ser encontrado o procesado correctamente.

4.2 Búsquedas

El sistema incorpora una funcionalidad de búsqueda que permite localizar países registrados dentro de la base de datos. Para ello, se recorre la lista principal de países mediante estructuras repetitivas, comparando el texto ingresado por el usuario con el nombre de cada país almacenado.

La búsqueda admite tanto coincidencias exactas como parciales. Esto significa que el usuario puede ingresar el nombre completo de un país o únicamente una parte de él. Por ejemplo, al ingresar "arg", el sistema es capaz de localizar el registro correspondiente a "Argentina".

Con el objetivo de mejorar la experiencia de uso, las comparaciones se realizan utilizando el método `lower()`, que convierte las cadenas de texto a minúsculas. Gracias a esta técnica, la búsqueda funciona correctamente independientemente de si el usuario escribe en mayúsculas, minúsculas o una combinación de ambas.

Cuando se encuentra una coincidencia, el sistema muestra toda la información asociada al país, incluyendo nombre, población, superficie y continente. En caso de no encontrar resultados, se informa al usuario mediante un mensaje claro

indicando que el país buscado no se encuentra registrado.

4.3 Filtros

La aplicación incorpora diferentes mecanismos de filtrado que permiten al usuario visualizar únicamente los países que cumplen determinados criterios. Esta funcionalidad facilita la consulta de información específica dentro del conjunto de datos almacenado. Se implementaron tres tipos de filtros principales:

Filtro por continente: permite seleccionar un continente determinado y obtener únicamente los países pertenecientes a dicha región. Para ello, el sistema presenta una lista de continentes disponibles y compara el continente elegido con el atributo correspondiente de cada país.

Filtro por población: permite definir valores mínimos y máximos para la cantidad de habitantes. El sistema recorre la lista de países y selecciona aquellos cuya población se encuentra dentro del rango especificado. Además, se contempla la posibilidad de ingresar únicamente un límite mínimo o máximo, ampliando la flexibilidad de la búsqueda.

Filtro por superficie: funciona de manera similar al filtro de población, permitiendo establecer rangos de superficie para identificar países con determinadas características territoriales.

Para realizar estos procesos, se recorren los registros almacenados mediante estructuras repetitivas y se construye una nueva lista con los países que cumplen los criterios seleccionados. Finalmente, los resultados se muestran en pantalla de forma organizada, indicando los datos principales de cada país encontrado.

En caso de que ningún registro coincida con los filtros aplicados, el sistema informa la situación mediante un mensaje claro para el usuario.

4.4 Ordenamientos

La aplicación incorpora funcionalidades de ordenamiento que permiten reorganizar los registros de países según distintos criterios. Esta característica facilita la consulta y el análisis de la información almacenada, permitiendo visualizar los datos de forma más organizada.

Se implementaron tres criterios de ordenamiento:

- **Nombre:** organiza los países alfabéticamente.
- **Población:** ordena los registros según la cantidad de habitantes.
- **Superficie:** organiza los países de acuerdo con su extensión territorial.

Para realizar el ordenamiento se utilizó la función `sorted()` de Python, generando una nueva lista ordenada sin modificar directamente la estructura original de datos. Además, mediante el parámetro `key`, se especifica el atributo que se utilizará como criterio de comparación en cada caso.

En el ordenamiento por nombre se emplea el método `lower()` para convertir los textos a minúsculas antes de compararlos, evitando diferencias provocadas por el uso de mayúsculas o minúsculas en los registros.

Asimismo, el sistema permite seleccionar entre orden ascendente o descendente mediante el parámetro `reverse`, brindando mayor flexibilidad al momento de visualizar la información.

Una vez aplicado el criterio elegido, los resultados se presentan en pantalla mostrando el nombre, población, superficie y continente de cada país, permitiendo una interpretación rápida y clara de los datos obtenidos.

4.5 Estadísticas

El sistema incorpora un módulo de estadísticas que permite obtener información relevante a partir del conjunto de países almacenados. Estas funciones proporcionan indicadores útiles para analizar los datos y realizar comparaciones entre los distintos registros.

Entre las estadísticas implementadas se encuentran la identificación del país con mayor población y del país con menor población. Para ello, se recorren todos los registros comparando la cantidad de habitantes de cada país hasta determinar los valores extremos.

Asimismo, se calcula el promedio de población y el promedio de superficie. Estos indicadores se obtienen mediante la suma de los valores correspondientes de todos los países y la posterior división por la cantidad total de registros almacenados.

Por otra parte, el sistema permite conocer la cantidad de países registrados en cada continente. Para esta tarea se utiliza un diccionario que almacena los continentes como claves y la cantidad de países asociados a cada uno como valor,

incrementando el contador correspondiente durante el recorrido de los datos.

Los resultados son presentados al usuario mediante mensajes claros y organizados, facilitando la interpretación de la información obtenida.

5 Validaciones y Manejo de Errores

Con el objetivo de garantizar la integridad de los datos y mejorar la experiencia del usuario, se implementaron diversas validaciones durante la interacción con el sistema. Estas verificaciones permiten detectar entradas incorrectas y solicitar nuevamente la información antes de continuar con la ejecución.

Para los campos de texto se desarrolló una función específica que impide el ingreso de valores vacíos, asegurando que información importante como el nombre de un país no quede sin completar.

En el caso de los datos numéricos, se implementó una validación que verifica que los valores ingresados sean números enteros positivos. Para ello se utilizó una estructura `try/except`, permitiendo capturar errores de conversión y mostrar mensajes adecuados cuando el usuario introduce caracteres no válidos.

También se incorporaron validaciones para las opciones de menú, comprobando que la alternativa seleccionada pertenezca al conjunto de opciones disponibles. De esta manera se evita la ejecución de acciones inexistentes dentro del sistema.

Por otra parte, se desarrolló una validación específica para los continentes, restringiendo los valores aceptados a América, Europa, Asia, África y Oceanía. Esto permite mantener consistencia en los datos almacenados y evita registros con nombres incorrectos o duplicados.

Además de las validaciones de entrada, el sistema contempla situaciones excepcionales durante la carga de datos desde archivos CSV, informando al usuario cuando el archivo no puede ser encontrado o procesado correctamente. Asimismo, las funciones de búsqueda y filtrado muestran mensajes informativos cuando no se encuentran resultados que coincidan con los criterios ingresados.

Estas medidas contribuyen a que la aplicación sea más robusta, reduciendo errores y garantizando un funcionamiento más confiable.

6 Evidencias de Funcionamiento

A continuación, se presentan capturas de pantalla representativas de la aplicación en ejecución, acompañadas de su respectiva descripción.

6.1 Agregar País

Permite incorporar un nuevo país al sistema ingresando su nombre, población, superficie y continente. Se aplican validaciones para evitar campos vacíos o datos inválidos.

```
===== MENU =====
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar
8. Salir
Seleccione una opción: 1
Nombre del país: Francia
Población: 65000000
Superficie: 675417
Continente: Europa
País agregado correctamente.
```

6.2 Actualizar País

Permite modificar la información de un país existente, actualizando los datos de población y superficie sin alterar el resto de la información registrada.

```
===== MENU =====
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar
8. Salir
Seleccione una opción: 2
Ingrese el nombre del país a actualizar: Argentina
País encontrado. Ingrese los nuevos datos:
Población actual (45376763): 46000000
Superficie actual (2780400): 2800000
País actualizado correctamente.
```

6.3 Buscar País

Permite localizar países mediante búsquedas exactas o parciales por nombre. La búsqueda no distingue entre mayúsculas y minúsculas, facilitando la consulta de información.

```
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar
8. Salir
Seleccione una opción: 3
Ingrese el nombre del país a buscar: Argentina

País encontrado:
Nombre: Argentina
Población: 46000000
Superficie: 2800000
Continente: America
```

6.3 Filtrar Países

Permite mostrar únicamente los países que cumplen determinados criterios, como pertenecer a un continente específico o encontrarse dentro de un rango de población o superficie.

```
Seleccione una opción: 4
.....Filtro de paises.....
Filtrar por:
1) Continente
2) Población
3) Superficie

Seleccione una opción: 1
Seleccione el continente:
1) America
2) Asia
3) Africa
4) Antartida
5) Europa
6) Oceania

Seleccione una opción: 5

.....RESULTADO DE FILTRO DE PAISES.....
- Datos filtrados por continente
- Continente elegido: Europa

País: Francia          | Población: 65000000   | Superficie: 675417    | Continente: Europa
```

6.4 Ordenar Países

Permite organizar los registros según distintos criterios (nombre, población o superficie) y elegir si el resultado se muestra en orden ascendente o descendente.

```
Seleccione una opción: 5
.....Ordenamiento de paises.....

Datos que definen el orden:
  1) Ordenar por nombre
  2) Ordenar por población
  3) Ordenar por superficie

Seleccione una opción: 1

Tipos de ordenamiento:
  1) Ascendente
  2) Descendente

Seleccione el orden: 2

.....RESULTADO DE ORDENAMIENTO DE PAISES.....
- Datos ordenados según su nombre
- Presentados en forma descendente

País: Japon      | Población: 125800000 | Superficie: 377975 | Continente: Asia
País: Francia   | Población: 65000000  | Superficie: 675417 | Continente: Europa
País: Brasil    | Población: 213993437 | Superficie: 8515767 | Continente: America
País: Argentina | Población: 46000000  | Superficie: 2800000 | Continente: America
```

6.5 Estadísticas

Calcula y presenta información relevante del conjunto de datos, incluyendo el país con mayor y menor población, los promedios de población y superficie, y la cantidad de países por continente.

```
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar
8. Salir
Seleccione una opción: 6

===== MENÚ DE ESTADÍSTICAS =====
1. País con mayor y menor población
2. Promedio de población
3. Promedio de superficie
4. Cantidad de países por continente
=====

Seleccione una opción: 1

===== MAYOR Y MENOR POBLACIÓN =====
País con mayor población: Brasil
Habitantes: 213993437

País con menor población: Argentina
Habitantes: 46000000
```

6.6 Guardar Datos

Permite almacenar los cambios realizados durante la ejecución del programa en el archivo CSV, conservando las modificaciones para futuras sesiones.

```
===== MENU =====
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Estadísticas
7. Guardar
8. Salir
Seleccione una opción: 7
Datos guardados correctamente.
```

7 Dificultades Encontradas

Durante el desarrollo del proyecto se identificaron diversas dificultades, las cuales se detallan a continuación junto con las estrategias empleadas para su resolución:

- **Organización modular del proyecto:** Inicialmente, integrar funciones dispersas generaba redundancia y confusión. Se resolvió estableciendo responsabilidades claras para cada módulo y utilizando importaciones estructuradas, mejorando la legibilidad y facilitando el trabajo colaborativo.
- **Trabajo colaborativo mediante Git y GitHub:** La coordinación de cambios entre integrantes presentó conflictos y problemas de sincronización. Se implementaron buenas prácticas como ramas temáticas, commits frecuentes y revisiones de código, logrando una integración fluida.
- **Integración de ramas:** Al fusionar diferentes versiones, no surgieron conflictos de código, ya que se utilizó una estrategia de revisión cuidadosa y comunicación constante entre integrantes.

- **Implementación de filtros y ordenamientos:** Definir criterios claros y eficientes para las operaciones sobre listas complejas requirió análisis y pruebas. Se optó por funciones integradas y funciones lambda para garantizar flexibilidad y rendimiento.
- **Persistencia de datos mediante CSV:** Se enfrentaron desafíos en el manejo correcto del encoding y formato para evitar errores de lectura y escritura. Se aplicaron codificaciones estándar y validaciones exhaustivas para asegurar la integridad de los datos almacenados.

Estos aprendizajes fortalecieron la capacidad técnica y el trabajo en equipo.

8 Conclusión

El desarrollo de este proyecto permitió aplicar y profundizar conocimientos fundamentales de programación, especialmente en el uso de estructuras de datos como listas y diccionarios, y en la modularización a través de funciones específicas. Se evidenció la importancia de mantener un código claro y segmentado para facilitar el mantenimiento y la escalabilidad.

Durante el proceso, se adquirió experiencia en el manejo de archivos CSV para la persistencia de datos, así como en la implementación de algoritmos para búsquedas, filtros, ordenamientos y estadísticas básicas, aspectos críticos en la gestión eficiente de información.

Además, la integración del trabajo colaborativo mediante Git y GitHub demostró ser esencial para coordinar tareas, controlar versiones y resolver conflictos, habilidades que son indispensables en entornos profesionales.

En síntesis, el proyecto contribuyó significativamente a consolidar competencias técnicas y de trabajo en equipo que serán valiosas para futuras actividades académicas y profesionales en programación.

9 Anexos

9.1 Enlace al repositorio GitHub

El código fuente del proyecto objeto del presente trabajo se encuentra disponible en el siguiente enlace:

<https://github.com/franck12A/tpi-gestion-paises-python.git>

9.2 Enlace al video demostración

Por otro lado, el video explicativo del proyecto en cuestión se puede encontrar en el siguiente enlace:

<https://www.youtube.com/watch?v=PK1TO6wFvto>

10. Referencias

McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.

Python Software Foundation. (2025). *Built-in Types — Dictionaries*. Python Documentation. <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

Python Software Foundation. (2025). *Built-in Types — Lists*. Python Documentation. <https://docs.python.org/3/library/stdtypes.html#lists>

Python Software Foundation. (2025). *Defining Functions*. Python Documentation. <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Python Software Foundation. (2025). *for Statements and while Statements*. Python Documentation. <https://docs.python.org/3/tutorial/controlflow.html>

Python Software Foundation. (2025). *More Control Flow Tools*. Python Documentation. <https://docs.python.org/3/tutorial/controlflow.html>